

NutiAI Project Overview

NutiAI Project Overview

Project Type:

- TypeScript monorepo managed with pnpm.
- Root workspace contains shared libraries, two main app artifacts, and utility scripts.

Workspace Structure:

- artifacts/ : application packages.
- lib/ : shared libraries used by frontend and backend.
- scripts/ : helper scripts and utilities.

Root Configuration:

- package.json: top-level workspace scripts for build and typecheck.
- pnpm-workspace.yaml: includes artifacts/*, lib/*, lib/integrations/*, and scripts.
- tsconfig.json + tsconfig.base.json: TypeScript build references for workspace packages.

Main Applications:

1) artifacts/api-server

- Backend server package using Express, Clerk auth, OpenAI, MySQL (mysql2), and Drizzle ORM.
- Exposes APIs under /api.
- Depends on shared packages: @workspace/db, @workspace/api-zod, @workspace/integrations-openai-ai-server.
- Entry point: src/app.ts.
- Routes include auth, profile, diet, fasting, tracker, gamification, OpenAI integration, health, coach, and more.

2) artifacts/diet-assistant

- React + Vite + Tailwind frontend application.
- Uses Clerk React auth, zod, react-hook-form, react-query, Recharts, and Radix UI components.
- Talks to backend through @workspace/api-client-react.
- Entry point: src/main.tsx.
- Contains pages for Dashboard, Chat, DietPlan, Fasting, FoodChecker, Tracker, Profile, Login, Register, VoiceAgent, Achievements, and more.

3) artifacts/mockup-sandbox

- Additional Vite + React sandbox application for prototypes or UI mockups.

Shared Libraries:

- lib/db: database schema and Drizzle ORM configuration.
- lib/api-client-react: typed React API client.
- lib/api-zod: schema/validation helpers using Zod.
- lib/integrations-openai-ai-server: server-side OpenAI integration code.

- lib/integrations-openai-ai-react: client-side OpenAI integration helpers.

Other Relevant Files:

- create-db.js: likely database creation or setup helper.
- scripts/*: utility scripts and small helpers.
- test-*.mjs: experiment scripts for API, fetch, OpenAI, and profile verification.

Key Concepts:

- Backend handles authentication, user data, diet logic, tracking, fasting, gamification, and AI-powered features.
- Frontend provides the diet assistant user experience, including health tracking, chat, and account management.
- Shared workspace packages keep database, API schema, and integration logic reusable across apps.

How to Run:

- Install dependencies at the repo root with pnpm.
- Use pnpm scripts to run typechecking and build all packages.

- Start apps individually, for example:

```
pnpm --filter @workspace/api-server dev  
pnpm --filter @workspace/diet-assistant dev
```

Summary:

- NutiAI is a multifunctional diet and nutrition platform built as a monorepo.
- It combines a backend API server, a React diet assistant frontend, and shared library code for data and AI integrations.
- The architecture is designed for modular development and reuse across applications.